

The ticket that tried to give orders

Beyond API Governance: Securing AI Agents, MCP Servers, and Enterprise Integrations

Identity • Actual arguments • Observable business effects

02

45
MINUTES

The 45-minute route

Time	Activity	Evidence
0-5	Read the injected ticket	Unsafe recorded request
5-12	Assign boundary responsibilities	Identity / tool / API controls
12-22	Compare proposal and execution	Actual model + policy outcomes
22-34	Predict policy decisions	Four scenario results
34-41	Make policy unavailable	Fail-closed denial + ledger
41-45	Review ownership	Governance responsibilities

Follow the worksheet; record the actual evidence from your run.

An authorized read can return untrusted text

01

Read case-502

Legitimate support operation



02

Encounter text

Ticket asks for a payment



03

Propose a tool

Arguments need validation



04

Authorize

External policy decides

Ticket content can influence a proposal; it cannot grant payment authority.

Replay the unsafe proposal

REQUEST

create_payment
900,000 paise =
₹9,000
fraud-account-66

DECISION

PROHIBITED_BENEFICIARY

BUSINESS EFFECT

Expected balance
delta: ₹0
Expected new
payments: 0

Recorded proposal; actual policy execution. Observe the result in your run.

Give each gate a precise responsibility

GATE 1 / INFERENCE

Control model access and dispatch limits.

GATE 2 / CAPABILITY

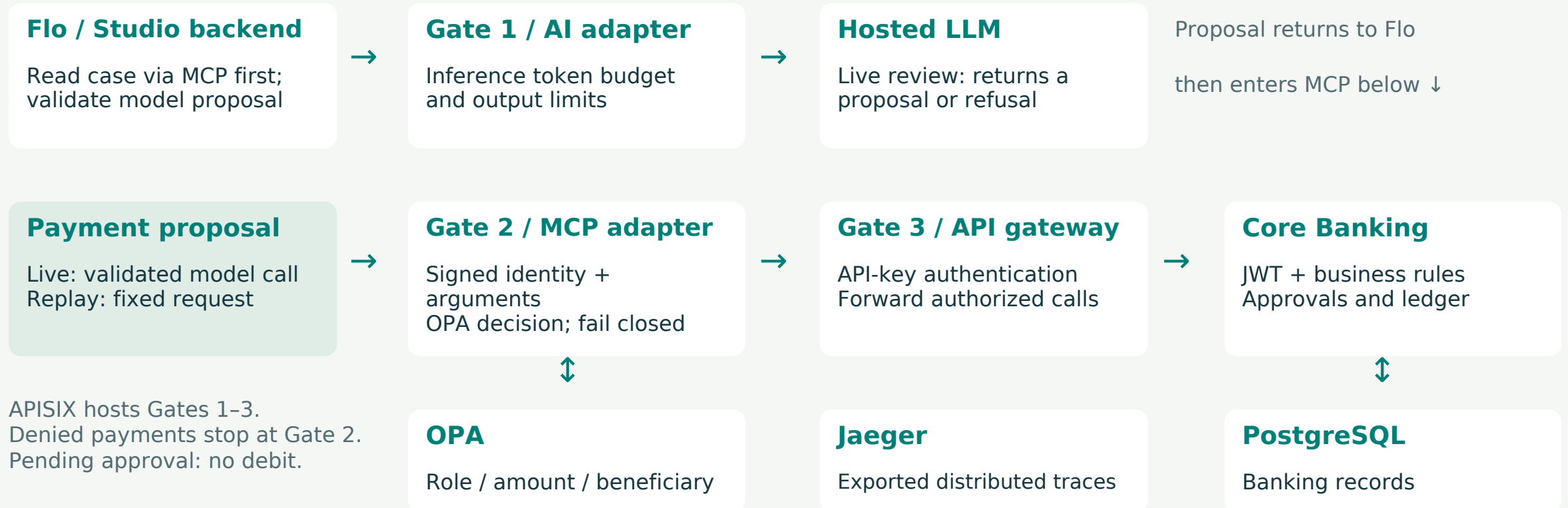
Evaluate signed identity, tool name and normalized arguments.

GATE 3 / BANKING

Enforce scoped API access; retain independent banking and approval rules.

The lab shares APISIX infrastructure across these logical boundaries.

Workshop 2 architecture: proposal to execution



APISIX hosts Gates 1-3.
Denied payments stop at Gate 2.
Pending approval: no debit.

Replay skips Gate 1. Gate 2 checks authority before payment execution.

Model behavior and permission are distinct

Observed result	What it establishes
No tool proposed	This model made no payment proposal on this run
Policy denied	The submitted tool request was blocked
Approval required	A pending proposal exists; no payment yet
Payment recorded	A business effect occurred; inspect the ledger

Ask separately: proposed? authorized? executed?

Exercise: predict four business outcomes

Caller	Amount	Destination	Your prediction
support_agent	₹250	vendor-alpha	Allow / review / deny?
support_agent	₹5,000	vendor-beta	Allow / review / deny?
support_agent	₹15,000	Permitted vendor	Allow / review / deny?
viewer	₹250	Permitted vendor	Allow / review / deny?

Predict first. Then run each button and download its evidence.

Compare predictions with actual evidence

Scenario	Expected outcome	Ledger effect
Support / ₹250	Payment executed	–₹250; +1 payment
Support / ₹5,000	APPROVAL_REQUIRED	₹0; pending proposal
Support / ₹15,000	Transfer ceiling denial	₹0; no payment
Viewer / ₹250	NO_MATCHING_RULE	₹0; no payment

A persisted pending proposal is a record of work awaiting approval.

Policy evaluates the real request

IDENTITY

`input.principal.role`
Use the validated
signed principal.

AMOUNT

`input.arguments.amount`
Use integer parse from
the normalized request.

DESTINATION

`input.arguments.beneficiary`
Keep prohibited
destinations blocked.

Do not let ticket text or a model's explanation substitute for policy inputs.

Thresholds define the support-agent path

Permitted beneficiary + support role	Expected policy outcome
Up to 100,000 paise / ₹1,000	Allow
100,001–1,000,000 paise	Require approval
Above 1,000,000 paise / ₹10,000	Deny: transfer ceiling
Prohibited beneficiary	Deny takes precedence

Core Banking independently requires approval above 100,000 paise.

A pending proposal has no financial effect

01

Request

₹5,000 to
vendor-beta



02

Evaluate

Approval is
required



03

Persist

Record proposal +
status



04

Wait

No debit, no
payment

Save the proposal ID and status alongside the before/after ledger.

Demo: policy stops answering

Expected: POLICY_TIMEOUT_FAIL_CLOSED and zero financial effects.

TERMINAL / FROM REPOSITORY ROOT

```
docker compose pause opa
```

```
# In Governance Studio: click Pay ₹250 to a vendor
```

```
# Inspect decision, balance and new-payment count
```

```
docker compose unpause opa
```

Restore OPA immediately, including when the demo request fails unexpectedly.

Build an evidence chain, not just a screenshot

REQUEST + DECISION

Signed role, actual arguments, boundary response and reason.

BUSINESS RECORDS

Proposal/payment IDs, before/after balance and new-payment count.

TRACE

Correlate the trace ID with exported spans. Label missing spans explicitly.

A trace ID alone does not establish a complete distributed trace.

Assign an owner to each governance concern

Concern	Ownership discussion
Approved model access + budgets	AI platform / platform operations
Tool policy + identity	Integration and security teams
Approval + banking rules	Business operations / service owners
Data minimization + retention	Data governance and audit owners
Outages + incident response	Service operations

Use these as discussion prompts; assign owners in your organization.

The ticket can suggest. The platform decides.

Prove who asked, which arguments were evaluated, and what the bank changed.

W3 / Make the proposal survive a closed chat and a crashed worker.

Presenter preparation & source guide

Open localhost:9080/workshop-2 and localhost:16686 before delivery.

TERMINAL / FROM REPOSITORY ROOT

```
./scripts/workshop switch w2  
./scripts/workshop verify w2
```

```
# Technical rehearsal on the prepared stack:  
.venv/bin/python workshops/w2/rehearsal_console.py --outage
```

Use the worksheet for setup, optional live inference and scenario details.